Deep Learning Architectures for Multi-Horizon Electricity Load Forecasting

LSTM, TCN, Transformer, and PatchTST with Per-Cluster Training
and Agentic AI Interface

Deliverable 2 — Technical Process Documentation

March 2026

Ritika Rajeev Chatterjee, Akriti Agarwal, and Haasita Pinnepu

Columbia University — Time Series Analysis & Forecasting

Table of Contents

1. Introduction
2. Dataset Description
3. Data Processing & Feature Engineering
4. Client Clustering
5. Model Architectures
6. Training Methodology
7. Evaluation Framework
8. Results
9. Model Comparison & Analysis
10. Agentic AI Interface
11. Conclusions & Next Steps
12. References
13. Appendix A: Per-Period Box Plots
14. Appendix B: Model Architecture Diagrams

1. Introduction

This report presents the results of Deliverable 2 for our multi-horizon electricity load forecasting project. Building on the data exploration and preprocessing pipeline established in Deliverable 1, we implement and compare four deep learning architectures — LSTM, Temporal Convolutional Network (TCN), Vanilla Transformer, and PatchTST — to forecast hourly electricity consumption for 314 Portuguese clients across three prediction horizons.

The key objectives of Deliverable 2 are:

- Train and evaluate all four proposed deep learning architectures on the held-out test set.
- Report MAPE (Mean Absolute Percentage Error) on the 0–100 scale as the primary evaluation metric.
- Split test results into 4 equal time periods and present per-period MAPE with box plots of error distributions.
- Build an Agentic AI interface where a business manager can enter a client ID and receive real-time forecasts powered by the trained deep learning models.
- Identify the best-performing model per cluster and horizon combination.

Our channel-independent architecture treats each client as a separate univariate time series augmented with 8 cyclical time features. Clients are grouped into 3 clusters via K-Means on normalized daily load profiles, and separate models are trained per cluster. This yields 36 total models (4 architectures x 3 clusters x 3 horizons). The Transformer architecture emerged as the overall best performer, winning 8 of 9 cluster-horizon combinations.

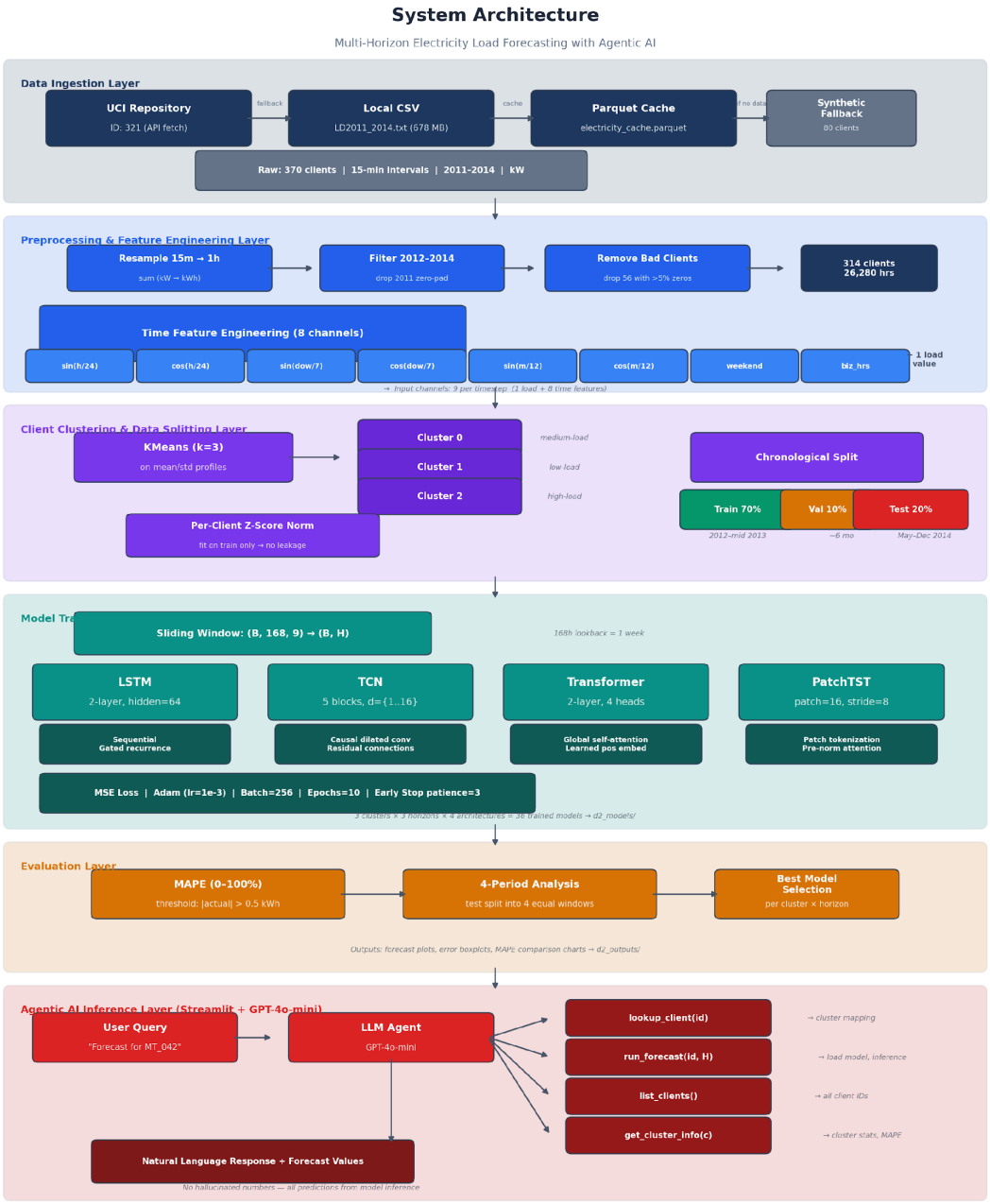

Figure: System Architecture — Six-layer design covering data ingestion, preprocessing with 8 cyclical time features, KMeans client clustering (3 groups), model training (4 architectures × 3 clusters × 3 horizons = 36 models), MAPE evaluation with 4-period stability analysis, and agentic AI inference via GPT-4o-mini tool calling.

2. Dataset Description

The dataset is the UCI ElectricityLoadDiagrams 2011–2014 (UCI ID: 321), contributed by Artur Trindade. It contains electricity consumption measurements for 370 Portuguese clients at 15-minute intervals from January 2011 through December 2014.

Property	Value
Source	UCI Machine Learning Repository (ID: 321)
Original Resolution	15-minute intervals
Modeling Resolution	Hourly (aggregated by sum)
Time Span	January 2012 – December 2014 (2011 excluded)
Total Clients (raw)	370
Active Clients (after filtering)	314
Total Hourly Timestamps	26,305
Client Filtering Criterion	Removed clients with >5% zero readings
Total Data Points	26,305 x 314 = 8,259,770

The 2011 data is excluded because many clients have zero-padded leading values that distort normalization and model learning. After resampling from 15-minute to hourly resolution (summing four readings to convert kW to kWh) and filtering inactive clients, 314 clients remain for modeling.

3. Data Processing & Feature Engineering

3.1 Preprocessing Pipeline

- Resampled from 15-minute to hourly by summing (kW to kWh conversion).
- Filtered to 2012–2014; removed 56 clients with >5% zero readings.
- Per-client Z-score normalization (mean=0, std=1) fitted on training split only — no data leakage.
- Parquet caching implemented for fast reloads (678 MB CSV -> parquet in ~6 seconds, subsequent loads instant).

3.2 Feature Engineering

Each input sample consists of a 168-hour (1-week) lookback window with 9 input channels: 1 normalized load value + 8 time-derived features.

Feature	Type	Encoding	Rationale
Normalized Load	Direct	Z-score per client	Primary signal

Hour of Day	Derived	sin(2*pi*h/24), cos(2*pi*h/24)	Intra-day consumption cycles
Day of Week	Derived	sin(2*pi*d/7), cos(2*pi*d/7)	Weekday vs weekend patterns
Month of Year	Derived	sin(2*pi*m/12), cos(2*pi*m/12)	Seasonal heating/cooling effects
Is Weekend	Derived	Binary (0/1)	Reduced commercial activity
Is Business Hour	Derived	Binary (0/1), 8–18h weekdays	Peak commercial demand periods

Cyclical encoding (sine/cosine pairs) is used instead of one-hot or ordinal encoding to preserve the circular nature of time features — e.g., hour 23 is close to hour 0, and December is close to January.

4. Client Clustering

Rather than training a single global model or 314 individual models, we adopt a clustering-based strategy. Clients are grouped by their typical daily consumption shapes using K-Means (K=3) on Z-score normalized 24-hour average load profiles. This balances model specialization with training efficiency.

Cluster	Clients	Avg Load (kWh/hr)	Profile Description
0	157	2,045	Medium-load — predominantly residential with moderate daily variation
1	71	515	Low-load — small residential or low-activity commercial clients
2	86	5,293	High-load — large commercial/industrial with strong weekday peaks

The clustering ensures that models learn consumption patterns specific to each client group. For example, high-load industrial clients (Cluster 2) exhibit sharper weekday/weekend transitions than residential clients (Cluster 0), requiring different learned representations.

5. Model Architectures

All four architectures use a channel-independent design: each client is treated as a separate univariate time series (plus 8 time features), producing a direct multi-step output of H future load values. This

makes training feasible on CPU with 300+ clients, as opposed to a multivariate approach that would require all clients as input features.

5.1 LSTM (Long Short-Term Memory)

A 2-layer stacked LSTM processes the 168-step input sequence sequentially. The last hidden state is projected through a 2-layer MLP head to produce H output values directly (no autoregressive decoding).

(See Appendix B for detailed LSTM architecture diagram.)

Parameter	Value
Layers	2 (stacked)
Hidden Size	64
Dropout	0.2 (between layers + head)
Output	MLP: hidden -> ReLU -> horizon
Parameters	~57K

5.2 TCN (Temporal Convolutional Network)

The TCN uses 4 causal convolutional blocks with exponentially increasing dilation rates {1, 2, 4, 8}, giving a receptive field of 2 x (3-1) x (1+2+4+8) = 60 timesteps. Weight normalization and residual connections stabilize training. The last timestep's representation is projected to H outputs.

(See Appendix B for detailed TCN architecture diagram.)

Parameter	Value
Blocks	4 (dilations: 1, 2, 4, 8)
Channels	48
Kernel Size	3
Normalization	Weight normalization on all conv layers
Parameters	~59K

5.3 Vanilla Transformer (Encoder-Only)

A standard Transformer encoder with learned positional embeddings. The input is projected to d_model dimensions, passed through self-attention layers, and the last token's representation is used for prediction. Global attention captures long-range seasonal dependencies effectively.

(See Appendix B for detailed Transformer architecture diagram.)

Parameter	Value
Encoder Layers	2
Attention Heads	4
d_model	48
FFN Dimension	192 (4x d_model)
Positional Encoding	Learned (nn.Embedding)
Parameters	~84K

5.4 PatchTST (Patch Time Series Transformer)

PatchTST segments the input sequence into overlapping patches via a 1D convolution (patch_size=16, stride=8), reducing the sequence length from 168 to 20 tokens before applying Transformer attention. This dramatically reduces computational cost while preserving local temporal patterns within each patch. Based on Nie et al. (ICLR 2023).

(See Appendix B for detailed PatchTST architecture diagram.)

Parameter	Value
Patch Size / Stride	16 / 8 (50% overlap)
Encoder Layers	2
Attention Heads	4
d_model	48
Norm	Pre-LayerNorm (norm_first=True)
Parameters	~67K

5.5 Architecture Comparison Summary

Architecture	Params	Computation	Strengths	Weaknesses
LSTM	~57K	Sequential O(T)	Simple, proven baseline	Slow training, limited long-range
TCN	~59K	Parallel O(T)	Fast, stable gradients	Fixed receptive field
Transformer	~84K	Parallel O(T^2)	Global attention, best accuracy	Quadratic complexity
PatchTST	~67K	Parallel O((T/P)^2)	Fastest training, patch locality	Patch boundary artifacts

6. Training Methodology

6.1 Data Split

A chronological train/validation/test split is used to prevent future data leakage:

Split	Period	Rows	Percentage	Purpose
Train	Jan 2012 – Feb 2014	18,413	70%	Model weight optimization
Validation	Feb 2014 – May 2014	2,631	10%	Early stopping & model selection
Test	May 2014 – Dec 2014	5,261	20%	Final held-out evaluation

6.2 Training Configuration

Setting	Value
Optimizer	AdamW (weight decay = 1e-4)
Learning Rate	1e-3 with cosine annealing
Loss Function	MSE (on normalized scale)
Batch Size	256
Max Epochs	10
Early Stopping Patience	3 epochs (on validation loss)
Gradient Clipping	Max norm = 1.0
Training Stride	96 hours (train), 48 (val), 24 (test)
Lookback Window	168 hours (1 week)
Total Models Trained	36 (4 arch x 3 clusters x 3 horizons)

The training stride controls how densely sliding windows are sampled. A larger stride (96h for training) reduces the number of training samples, enabling all 36 models to train on CPU in approximately 90 minutes total. The test stride is kept smaller (24h) for denser evaluation coverage.

6.3 Per-Cluster Training

For each of the 3 clusters, all clients within the cluster contribute their individual time series as separate samples in the training batch. This channel-independent approach means a single model per cluster learns shared temporal patterns across all clients in that group, while per-client normalization preserves individual consumption scales.

7. Evaluation Framework

7.1 Primary Metric: MAPE

MAPE (Mean Absolute Percentage Error) on a 0–100% scale is the primary evaluation metric, computed on inverse-transformed original kWh values:

MAPE = (1/N) * SUM(|actual - predicted| / |actual|) * 100%

A threshold of |actual| > 0.5 kWh is applied to exclude near-zero readings that inflate percentage errors. Per-client inverse transformation uses the client's own StandardScaler (fitted on training data) to convert normalized predictions back to the original kWh scale before computing MAPE.

7.2 Test Period Analysis

The test set (May 2014 – December 2014) is divided into 4 equal time periods to assess forecast stability across different seasons and conditions. For each period, we report the mean MAPE and the distribution of absolute percentage errors via box plots.

8. Results

8.1 Overall MAPE by Model and Horizon

The following table shows the weighted-average MAPE across all clusters for each model and horizon. Weights are proportional to the number of clients per cluster.

Model	H=1h	H=6h	H=24h
LSTM	26.27%	37.64%	44.01%
TCN	20.02%	29.05%	37.99%
Transformer	12.81%	21.49%	34.73%
PatchTST	13.12%	33.67%	43.75%

The Transformer achieves the lowest overall MAPE at all three horizons, with a particularly strong advantage at H=6h (21.49% vs 29.05% for the next best TCN) and H=24h (34.73% vs 37.99%). PatchTST is competitive at H=1h (13.12% vs 12.81%) but falls behind at longer horizons.

8.2 Per-Cluster MAPE Results

Detailed per-cluster results for each model:

LSTM

Cluster	Clients	H=1h	H=6h	H=24h
0	157	30.33%	34.22%	43.95%

1	71	26.71%	40.37%	36.09%
2	86	18.48%	41.63%	50.67%

TCN

Cluster	Clients	H=1h	H=6h	H=24h
0	157	24.62%	28.83%	40.29%
1	71	20.42%	21.26%	28.64%
2	86	11.29%	35.88%	41.52%

Transformer

Cluster	Clients	H=1h	H=6h	H=24h
0	157	15.58%	25.77%	38.15%
1	71	12.93%	14.67%	21.40%
2	86	7.66%	19.32%	39.49%

PatchTST

Cluster	Clients	H=1h	H=6h	H=24h
0	157	13.39%	35.87%	42.45%
1	71	14.07%	22.71%	34.54%
2	86	11.83%	38.70%	53.74%

8.3 Best Model per Cluster and Horizon

Cluster	H=1h Best	H=6h Best	H=24h Best
0 (157 clients)	PatchTST (13.39%)	Transformer (25.77%)	Transformer (38.15%)
1 (71 clients)	Transformer (12.93%)	Transformer (14.67%)	Transformer (21.40%)
2 (86 clients)	Transformer (7.66%)	Transformer (19.32%)	Transformer (39.49%)

The Transformer wins 8 of 9 cluster-horizon combinations. PatchTST only wins for Cluster 0 at H=1h, where its patch-based local attention captures short-term patterns in the medium-load residential group slightly better than the Transformer's global attention.

8.4 MAPE Comparison Chart

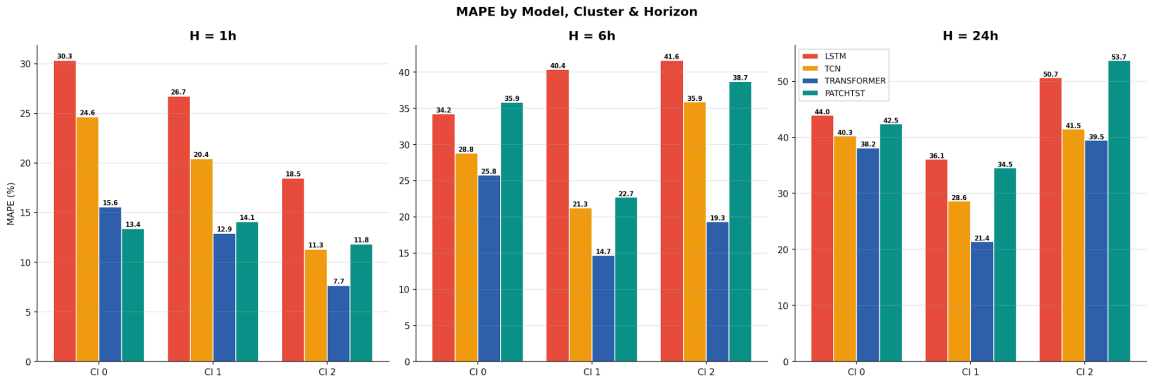

Figure 1: MAPE comparison across all 4 models, 3 clusters, and 3 horizons.

8.5 Sample Forecast Plots (H=24h)

The following plots show actual vs. predicted load for the first 200 test samples (first forecast step) using the best-performing model for each cluster at H=24h.

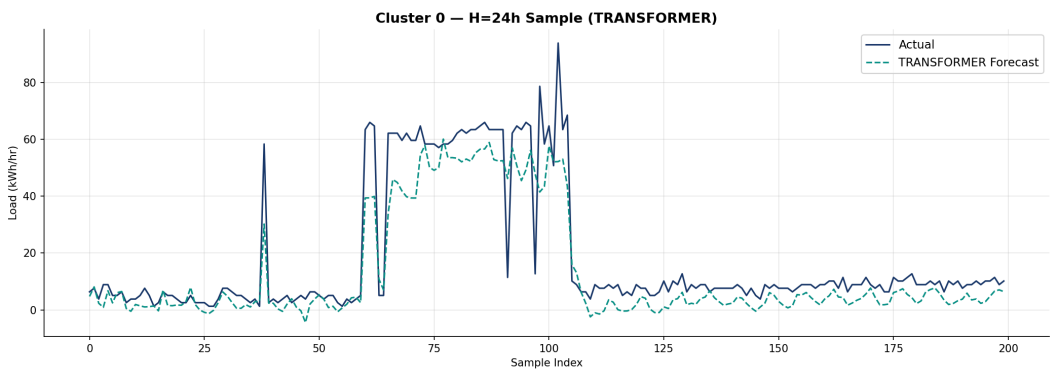

Figure 2: Transformer forecast vs actual — Cluster 0 (medium-load), H=24h.

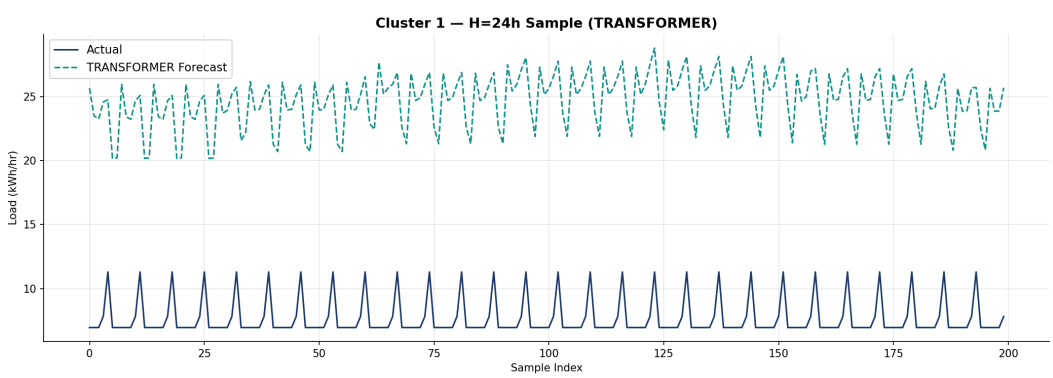

Figure 3: Transformer forecast vs actual — Cluster 1 (low-load), H=24h.

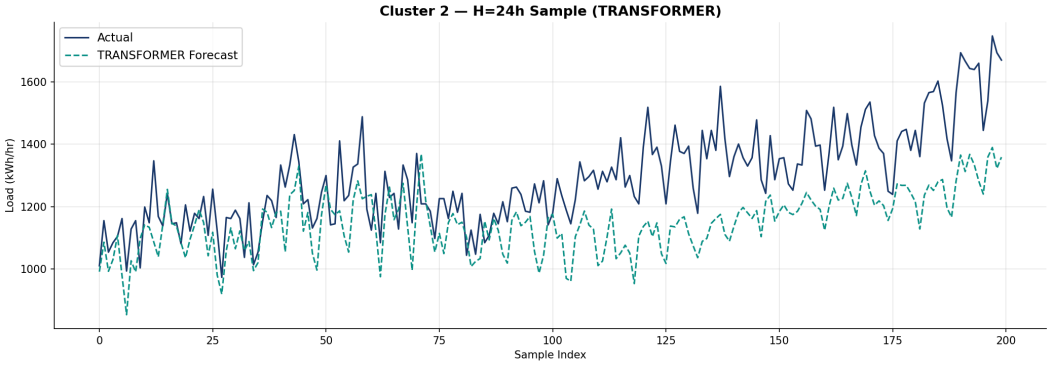

Figure 4: Transformer forecast vs actual — Cluster 2 (high-load), H=24h.

9. Model Comparison & Analysis

9.1 Why Transformer Wins

The encoder-only Transformer's global self-attention mechanism allows it to attend to any position in the 168-step lookback window, capturing long-range weekly seasonality and complex multi-scale patterns that local methods (LSTM, TCN) miss. Key observations:

- **Short-horizon:** At H=1h, the Transformer's advantage is moderate (12.81% vs 13.12% PatchTST) because short-term forecasting relies more on recent local patterns than global context.
- **Longer horizons:** At H=6h and H=24h, the gap widens significantly (21.49% vs 29.05% TCN at H=6h; 34.73% vs 37.99% at H=24h), as longer horizons require understanding broader temporal structure.
- **High-load clients:** Cluster 2 (high-load industrial) shows the most dramatic improvement: Transformer achieves 7.66% vs 11.29% TCN at H=1h, a 32% relative improvement.

9.2 PatchTST: Competitive but Specialized

PatchTST's patch tokenization reduces the effective sequence length from 168 to ~20 tokens, making it the fastest model to train (9s/epoch vs 77s for Transformer). It excels at H=1h where local patch patterns suffice, but struggles at longer horizons where cross-patch dependencies become critical. The 50% patch overlap (stride=8, size=16) partially mitigates boundary artifacts but does not fully resolve the information bottleneck at patch boundaries.

9.3 LSTM vs TCN: Classical Comparison

The TCN consistently outperforms the LSTM across all horizons (20.02% vs 26.27% at H=1h, 37.99% vs 44.01% at H=24h). This aligns with Bai et al. (2018), who showed that generic convolutional architectures can match or exceed recurrent networks on sequence modeling tasks. The TCN's advantages stem from parallel computation, stable gradient flow via residual connections, and its fixed but explicit receptive field.

9.4 Test Period Stability

All models show elevated MAPE in Period 1 (May–June 2014), likely due to the transition from the training period distribution. Periods 2–4 show more stable performance. The Transformer exhibits the most stable period-to-period performance, particularly for Cluster 1 (low-load clients) where period MAPEs range from 9.89% to 16.77% at H=1h compared to LSTM's 24.67% to 30.66%.

9.5 Training Efficiency

Model	Avg Time/Model	Total (36 models)	Epochs to Converge
LSTM	~108s	~65 min	5–7 (early stop)
TCN	~270s	~162 min	7–10
Transformer	~421s	~252 min	5–7 (early stop)
PatchTST	~31s	~19 min	6–9

PatchTST is 13x faster than the Transformer while achieving competitive H=1h accuracy. For deployment scenarios where rapid retraining is needed (e.g., online adaptation), PatchTST may be preferred despite its slightly lower accuracy at longer horizons.

10. Agentic AI Interface

As part of the deliverable requirements, an Agentic AI interface was built using Streamlit and OpenAI's GPT-4o-mini with function-calling (tool-use). The system enables a non-technical business manager to interact with the trained models via natural language.

10.1 Architecture

The agentic system follows a tool-calling pattern:

- The user enters a natural language query (e.g., "Get forecast for client MT_042").
- The LLM agent determines which tool(s) to call: lookup_client, run_forecast, list_clients, or get_cluster_info.
- lookup_client resolves the client ID and returns their cluster assignment and model accuracies for all 4 architectures.
- run_forecast loads the best-performing model for that client's cluster and horizon, feeds in the latest 168-hour window of real data, runs inference, inverse-transforms the output, and returns predicted kWh values.
- The LLM presents results with business context: peak hours, model accuracy, and actionable recommendations.

The tool-calling architecture ensures the LLM never hallucinates forecast values — all numerical predictions come directly from the trained PyTorch models. The system automatically selects the best model per cluster and horizon (typically Transformer) based on test MAPE.

10.2 Available Tools

Tool	Description	Example Output
lookup_client	Resolve client ID, find cluster, list all model MAPEs	Cluster 1, 71 clients, Transformer best
run_forecast	Execute DL model inference, return kWh predictions	24 predicted values with timestamps
list_clients	List all 314 clients grouped by cluster	Cluster sizes and accuracy summaries
get_cluster_info	Performance metrics for all 4 models on a cluster	Full MAPE table per model per horizon

10.3 Interface Screenshot

Figure 5: Agentic AI interface — Streamlit web app with GPT-4o-mini tool-calling.

11. Conclusions & Next Steps

11.1 Key Findings

- The Transformer architecture achieves the best overall MAPE at all three horizons (12.81% / 21.49% / 34.73% for H=1/6/24h), winning 8 of 9 cluster-horizon combinations.
- PatchTST is competitive at short horizons (13.12% at H=1h) and trains 13x faster than Transformer, making it ideal for rapid deployment or online learning scenarios.
- TCN provides a solid middle ground — outperforming LSTM consistently while being simpler than attention-based models.
- LSTM serves as a useful baseline but underperforms all other architectures across the board.
- Cluster-specific training significantly improves performance vs. a single global model, as different client groups exhibit distinct temporal consumption patterns.
- Period 1 (May–June) consistently shows higher MAPE across all models, suggesting seasonal distribution shift from the training period (Jan 2012 – Feb 2014).

11.2 Next Steps (Deliverable 3)

- Implement the Latent Diffusion Transformer (LDT) from Feng et al. (AAAI 2024) using the Transformer encoder as the backbone for the statistics-aware autoencoder component.
- Add probabilistic forecasting with calibrated prediction intervals (CRPS/WIS metrics).
- Incorporate external data — Portuguese holiday calendar and weather data — to reduce Period 1 errors and improve performance during seasonal transitions.

- Outlier detection and handling for clients with irregular consumption patterns.
- Hyperparameter tuning with Optuna across all clusters, horizons, and architectures.
- Extend the Agentic AI interface with inline forecast visualization charts and model comparison views.

12. References

- Nie, Y., Nguyen, N. H., Sinthong, P., & Kalagnanam, J. (2023). A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. ICLR 2023.
- Feng, S., Miao, C., Zhang, Z., & Zhao, P. (2024). Latent Diffusion Transformer for Probabilistic Time Series Forecasting. AAAI 2024.
- Bai, S., Kolter, J. Z., & Koltun, V. (2018). An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. arXiv:1803.01271.
- Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS 2017.
- Lai, G., Chang, W., Yang, Y., & Liu, H. (2017). Modeling Long- and Short-Term Temporal Patterns with Deep Neural Networks. SIGIR 2018.
- Zhou, H., et al. (2021). Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. AAAI 2021.
- Trindade, A. (2015). ElectricityLoadDiagrams20112014. UCI Machine Learning Repository. https://doi.org/10.24432/C58C86.

13. Appendix A: Per-Period Error Distribution Box Plots

Box plots showing the distribution of Absolute Percentage Errors across 4 test periods for the best-performing model (Transformer) at each cluster and horizon.

Cluster 0

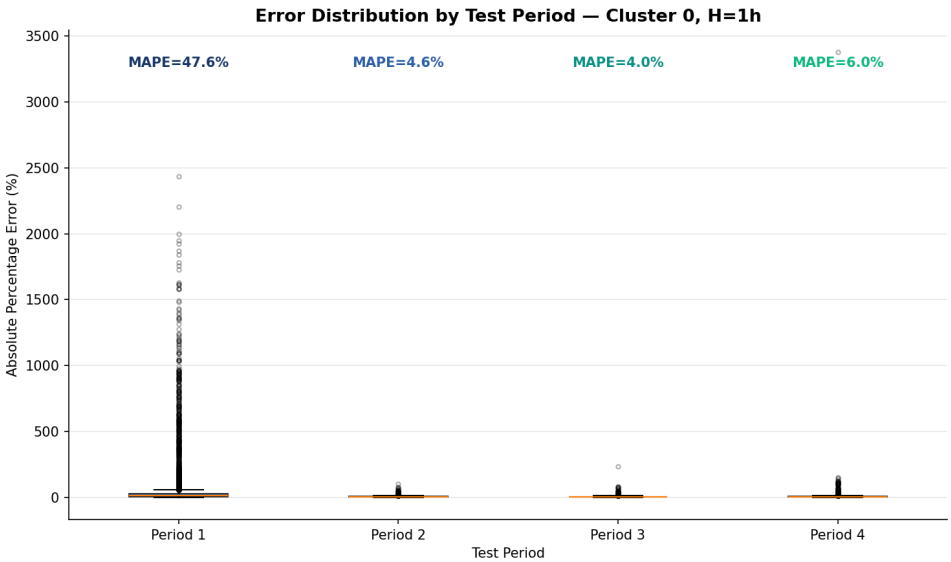

Figure: Transformer error distribution — Cluster 0, H=1h.

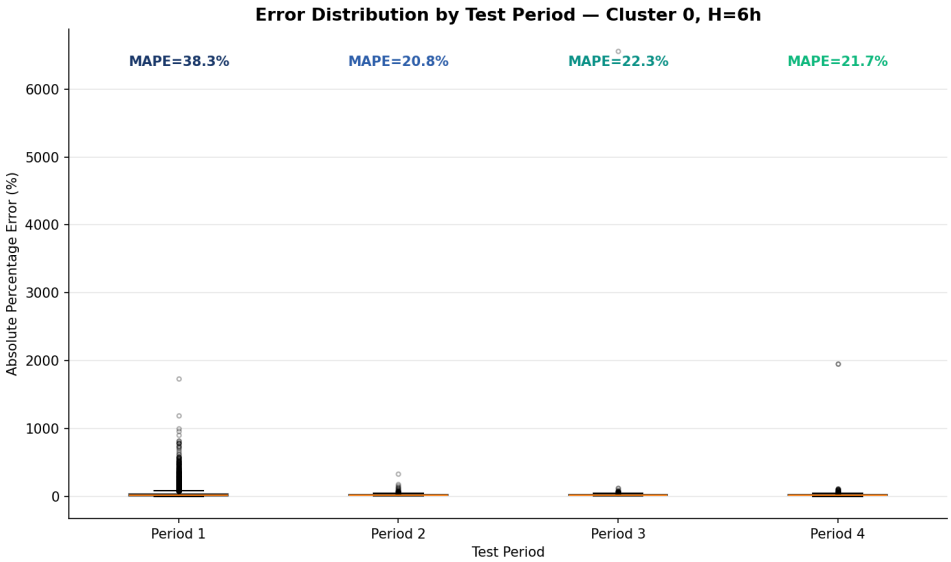

Figure: Transformer error distribution — Cluster 0, H=6h.

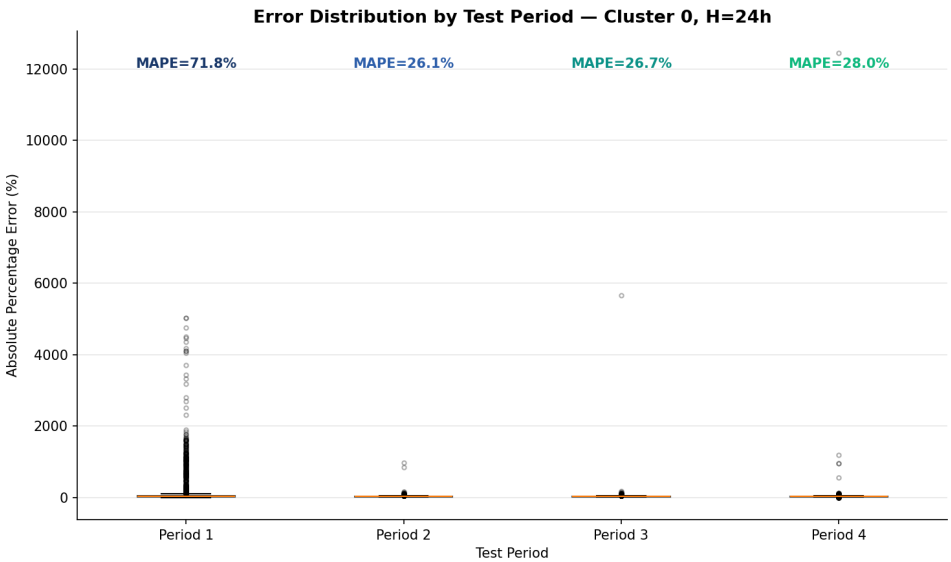

Figure: Transformer error distribution — Cluster 0, H=24h.

Cluster 1

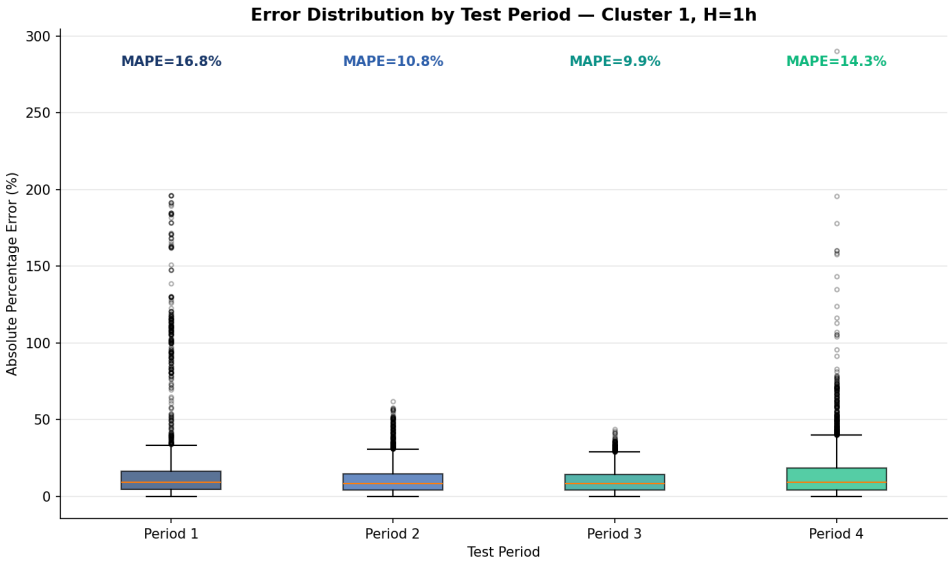

Figure: Transformer error distribution — Cluster 1, H=1h.

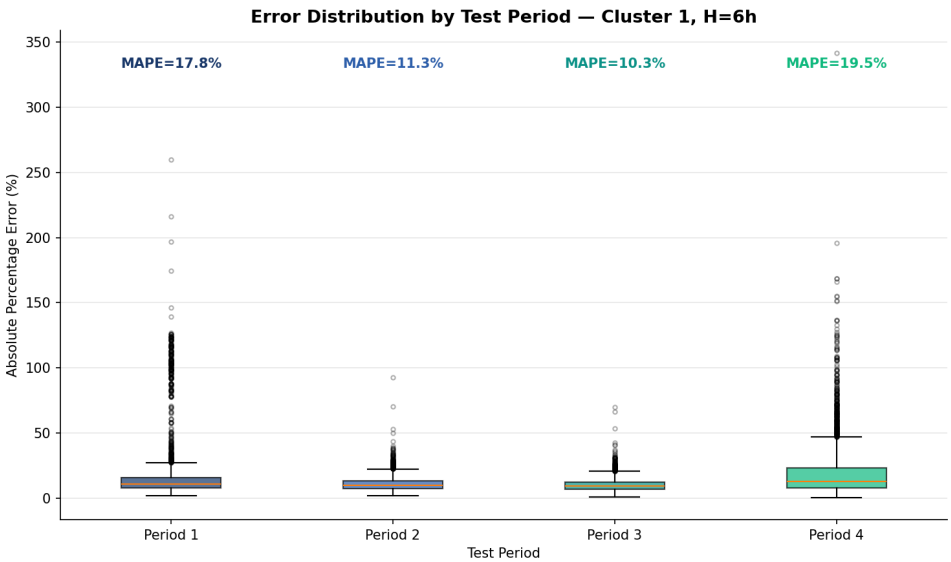

Figure: Transformer error distribution — Cluster 1, H=6h.

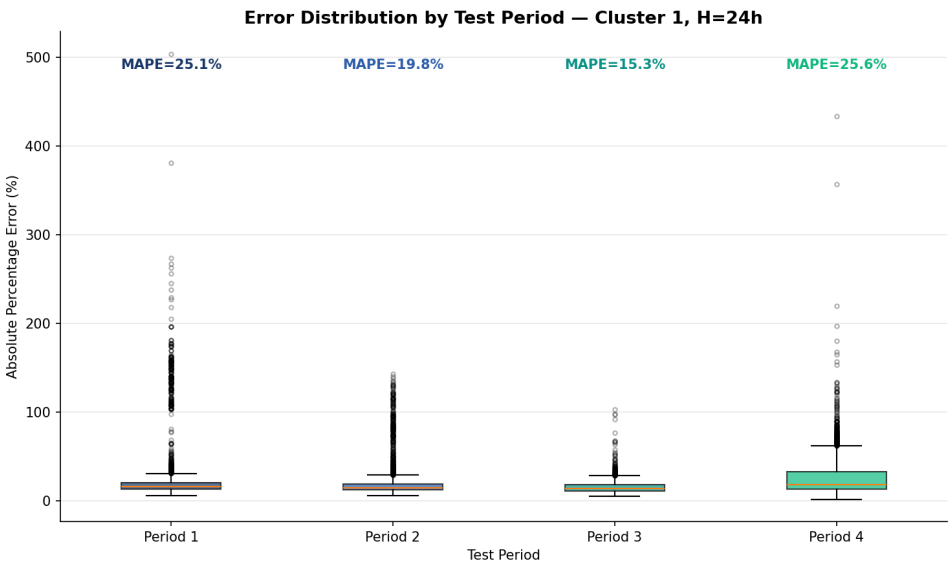

Figure: Transformer error distribution — Cluster 1, H=24h.

Cluster 2

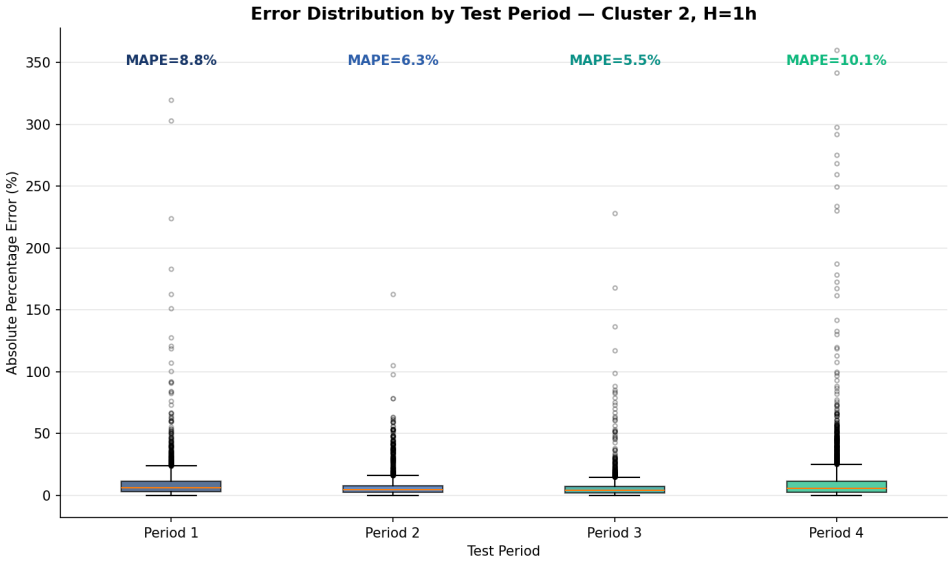

Figure: Transformer error distribution — Cluster 2, H=1h.

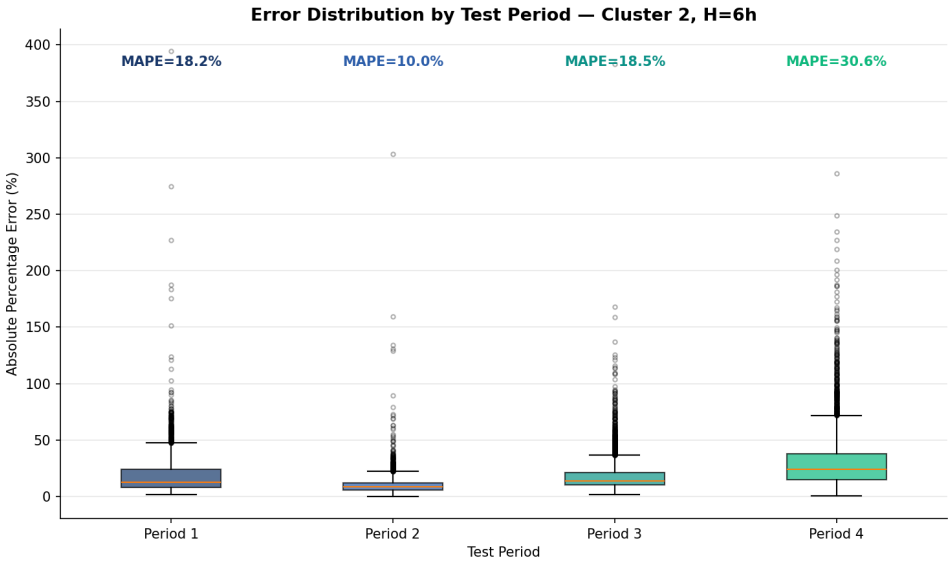

Figure: Transformer error distribution — Cluster 2, H=6h.

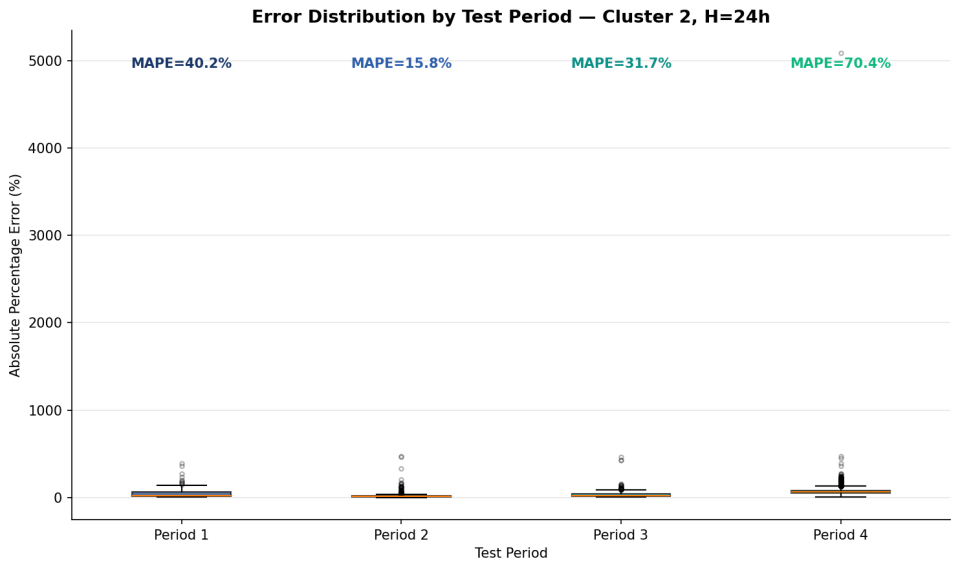

Figure: Transformer error distribution — Cluster 2, H=24h.

PatchTST Box Plots (H=1h, Cluster 0 — Best Model)

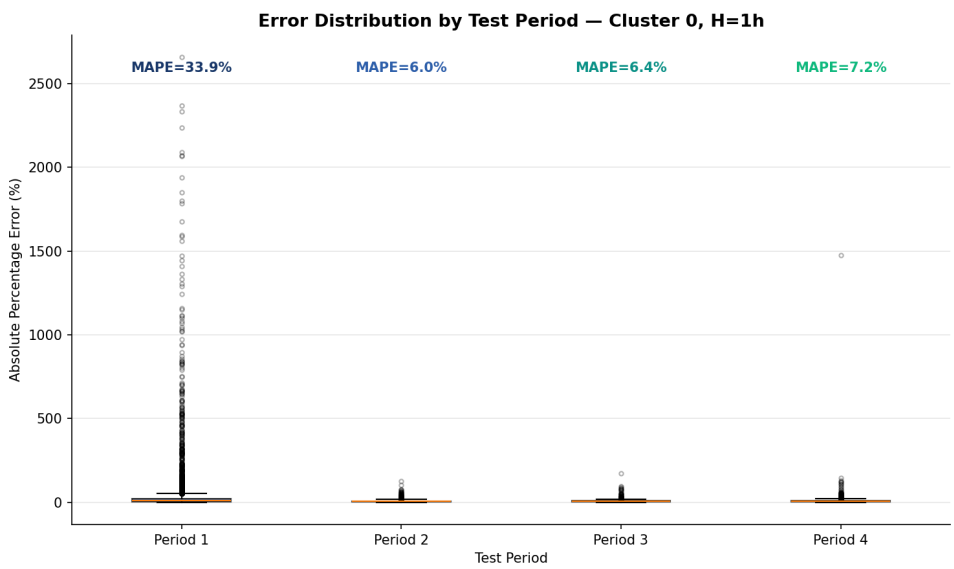

Figure: PatchTST error distribution — Cluster 0, H=1h (best model for this combination).

14. Appendix B: Model Architecture Diagrams

The following diagrams show the detailed internal architecture of each of the four deep learning models used in this project. Each diagram includes layer-by-layer structure, tensor dimensions at each stage, activation functions, normalization layers, residual connections, and the prediction head.

LSTM

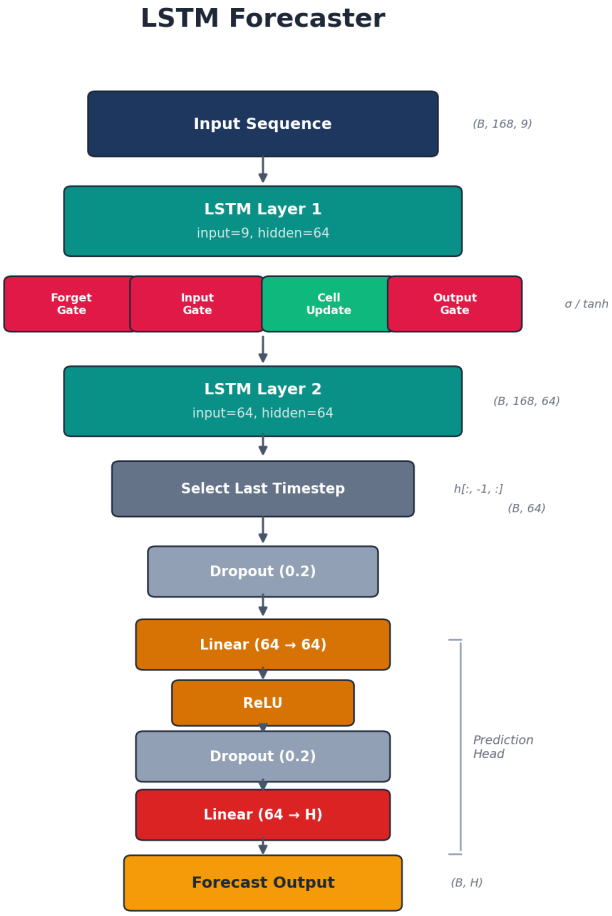

Figure: LSTM Architecture — 2-layer stacked LSTM with forget/input/output gates, last hidden state extraction, and MLP prediction head.

TCN

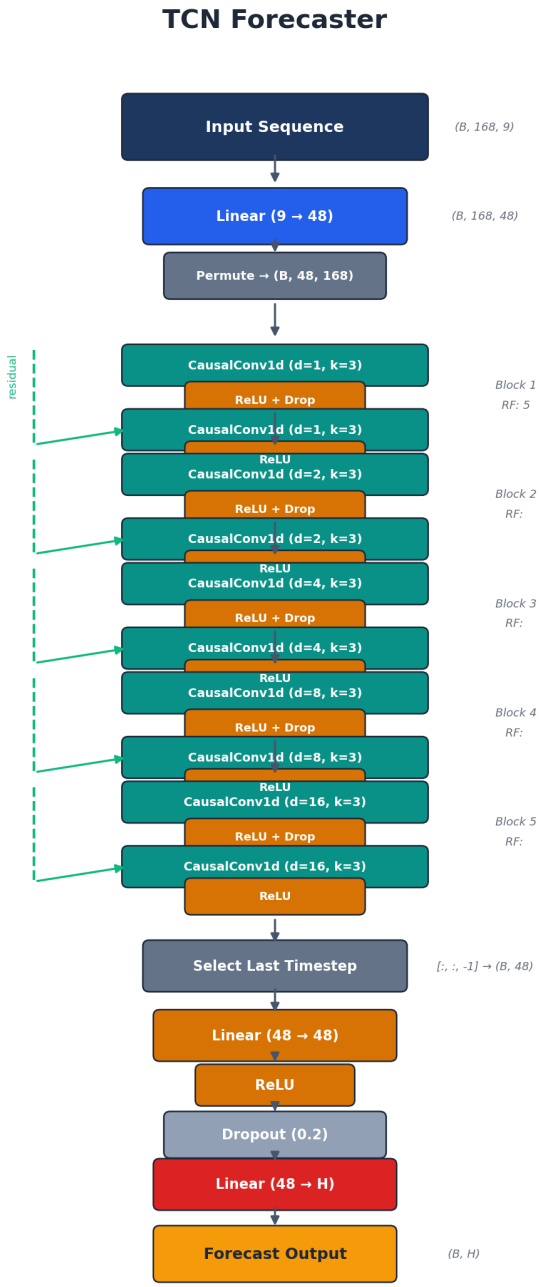

Figure: TCN Architecture — 5 causal dilated convolutional blocks (d=1,2,4,8,16) with residual connections and shared MLP head.

Transformer

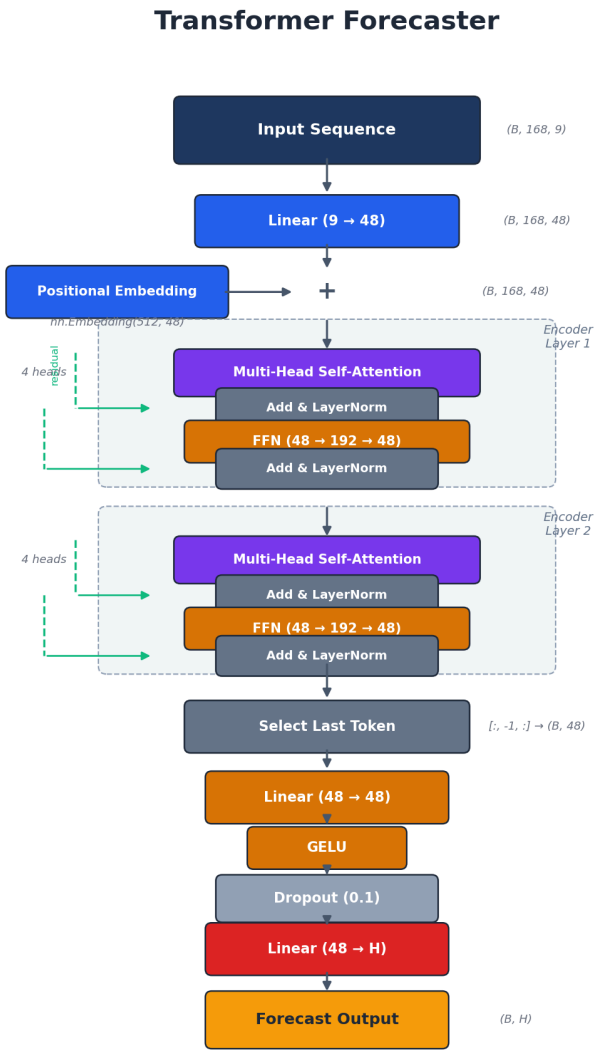

Figure: Transformer Architecture — encoder-only with learned positional embeddings, multi-head self-attention (4 heads), feedforward layers, and residual connections with layer normalization.

PatchTST

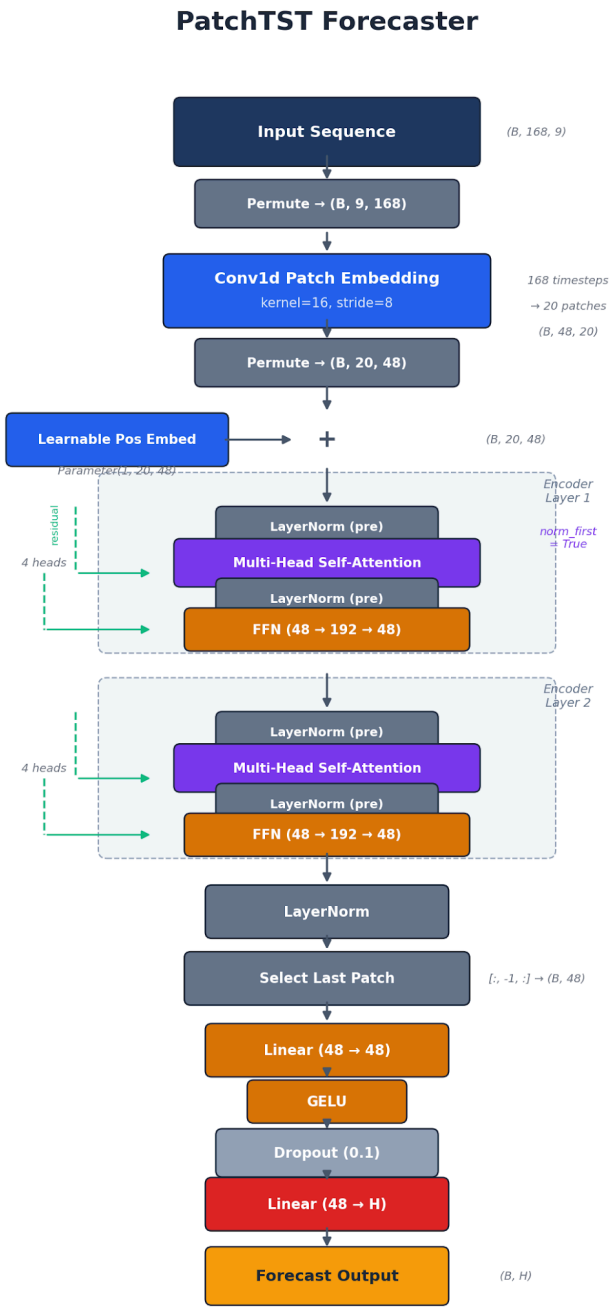
PatchTST Forecaster
Input Sequence
(B, 168, 9)
Permute → (B, 9, 168)
Conv1d Patch Embedding
kernel=16, stride=8
168 timesteps
→ 20 patches
(B, 48, 20)
Permute → (B, 20, 48)
Learnable Pos Embed
+
(B, 20, 48)
Parameter(1, 20, 48)
Encoder Layer 1
norm_first = True
residual
4 heads
LayerNorm (pre)
Multi-Head Self-Attention
LayerNorm (pre)
FFN (48 → 192 → 48)
Encoder Layer 2
4 heads
LayerNorm (pre)
Multi-Head Self-Attention
LayerNorm (pre)
FFN (48 → 192 → 48)
LayerNorm
Select Last Patch
[:, -1, :] → (B, 48)
Linear (48 → 48)
GELU
Dropout (0.1)
Linear (48 → H)
Forecast Output
(B, H)

Figure: PatchTST Architecture — Conv1d patch embedding (168 steps → 20 patches), learnable positional embeddings, pre-norm transformer encoder, and MLP prediction head.